# xgboost_timewindow

## 环境安装、配置与启用指南

Environment Installation, Configuration and Activation Guide

| | |
|---|---|
| 文档版本 | V2.0（结构调整稿） |
| 适用软件 | xgboost_timewindow |
| 适用平台 | Linux x86_64、C Shell、集群运行环境 |
| 编制日期 | 2026 年 8 月 22 日 |
| 文档状态 | 新版副本；不替代原环境启用说明 |

# 文档控制

## 修订记录

| 文档版本 | 日期 | 适用软件版本 | 修改说明 |
| --- | --- | --- | --- |
| V2.0 | 2026 年 8 月 22 日 | 当前工作区版本 | 按软件指南结构新建副本；重组安装、配置、操作、验证和故障排查内容。 |

## 文档约定

本文中的路径均使用变量表示，不把某台服务器上的绝对路径作为可移植配置。命令示例默认从 Bash/Sh 发起；若在 C Shell/Tcsh 中交互执行，应使用 `setenvNAMEvalue` 设置环境变量。

正文中的“必须”表示缺少该条件时流程不能按预期运行；“建议”表示有助于复现、排查或控制资源，但不一定阻止程序启动。

# 目录

**文档控制** **1**

**第一章 概述** **4**
1.1 编写目的 . . . . . 4
1.2 适用范围 . . . . . 4
1.3 目标读者 . . . . . 4
1.4 软件组成与文档边界 . . . . . 4

**第二章 使用场景与快速开始** **6**
2.1 场景选择 . . . . . 6
2.2 已有 cache 的最短验证路径 . . . . . 6
2.3 完整启用顺序 . . . . . 7

**第三章 安装与部署** **8**
3.1 基础要求 . . . . . 8
3.2 Python 环境部署 . . . . . 8
3.2.1 方式一：使用已有环境 . . . . . 8
3.2.2 方式二：迁移 conda-pack 环境 . . . . . 8
3.2.3 方式三：重新创建环境 . . . . . 9
3.3 Python 依赖验证 . . . . . 9
3.4 FSDBAPI 与 FsdbReader 部署 . . . . . 10

**第四章 配置** **11**
4.1 路径模型 . . . . . 11
4.2 训练链路必填配置 . . . . . 11
4.3 训练调度配置 . . . . . 12
4.4 FSDB 链路配置 . . . . . 12
4.5 站点激活文件 . . . . . 12

**第五章 操作指南** **14**
5.1 启动已有 cache 训练 . . . . . 14
5.2 选择代理筛选方式 . . . . . 14
5.3 启动 FSDB 前处理 . . . . . 15

5.4 查询运行状态 . . . . . 15
5.5 续跑与重新运行 . . . . . 16

**第六章 验证与验收** **17**
6.1 L0：静态环境检查 . . . . . 17
6.2 L1：输入与权限检查 . . . . . 17
6.3 L2：小规模端到端验证 . . . . . 18
6.4 验收标准 . . . . . 18

**第七章 故障排查** **19**
7.1 Python 解释器或依赖失败 . . . . . 19
7.2 MCP 结果与预期不一致 . . . . . 19
7.3 FsdbReader 动态库失败 . . . . . 19
7.4 Verdi license 失败 . . . . . 19
7.5 调度任务提交失败 . . . . . 20
7.6 输入数据或 namefile 不匹配 . . . . . 20
7.7 已有产物被错误复用 . . . . . 20
7.8 内存或磁盘不足 . . . . . 20

**附录 A 常用环境变量参考** **21**
A.1 训练路径与输入 . . . . . 21
A.2 常用训练控制变量 . . . . . 21

**附录 B 命令速查** **23**
B.1 Python 依赖检查 . . . . . 23
B.2 快速训练验证 . . . . . 23
B.3 正式训练 . . . . . 23
B.4 FSDB 前处理 . . . . . 23

**附录 C 术语与缩略语** **24**

# 第一章　概述

## 1.1　编写目的

本文档用于指导安装、迁移、配置和验证 `xgboost_timewindow` 的运行环境，使使用者能够完成以下任务：

- 使用已有降维 cache 启动 time-window 训练；
- 从原始 FSDB 启动特征导出、标签提取和降维；
- 验证 Python、FsdbReader、Verdi 和集群调度环境；
- 根据日志、状态文件和常见症状定位环境问题。

## 1.2　适用范围

本文档适用于 Linux x86_64 服务器、C Shell/Tcsh 脚本环境以及当前工程使用的 `bsub`、`dsub` 集群提交方式。本文档说明环境和操作接口，不展开 Step1–Step6 算法、MCP/XGBoost 原理、数据 shape 演化或 FSDBAPI 内部接口实现。

## 1.3　目标读者

| 角色 | 关注内容 |
|---|---|
| 环境管理员 | Python/Conda 环境迁移、Verdi/FsdbReader、license、编译器和调度器。 |
| 训练使用者 | 输入 cache、训练变量、启动命令、运行状态和结果目录。 |
| 开发维护者 | FSDBAPI 重新编译、依赖变更、环境故障和文档版本维护。 |

## 1.4　软件组成与文档边界

| 目录或文档 | 职责 |
|---|---|
| `cmd/` | FSDB 前处理、time-window 训练、单步运行和验证入口。 |
| `src/timewindow/` | Step1–Step6 的 Python 实现。 |

| | |
|---|---|
| src/reduction/ | Level/TOG cache 降维实现。 |
| src/fsdbapi/ | FSDB 导出程序、Makefile、FsdbReader 依赖与接口说明。 |
| docs/timewindow_training_workflow.md | 训练数据流、算法步骤和参数说明。 |
| src/fsdbapi/fsdbapi_Instruction.md | FSDBAPI 编译、接口和数据处理细节。 |

# 第二章　使用场景与快速开始

## 2.1　场景选择

| 场景 | 建议入口 | 需要阅读的章节 |
| --- | --- | --- |
| 已有 Python 环境和降维 cache | 训练验证脚本 | 第 2、4、5、6 章 |
| 需要迁移已验证的 Conda 环境 | conda-pack | 第 3、4、6 章 |
| 从原始 FSDB 生成 cache | FSDB pipeline | 第 3、4、5、6 章 |
| 新服务器完整部署 | 完整流程 | 第 3 章至第 7 章 |

## 2.2　已有 cache 的最短验证路径

**前置条件：**目标机器已存在可用的 Python 解释器、level/tog cache、power label 和匹配的 namefile。

在 Bash/Sh 中执行：

```
cd /path/to/xgboost_timewindow

PYTHON=/path/to/conda/envs/pro/bin/python \
LEVEL_DIR=/path/to/reduced/level \
TOG_DIR=/path/to/reduced/tog \
LABEL_DIR=/path/to/power_labels \
NAMEFILE=/path/to/reduced/level/after_one.namefile.txt \
OUTPUT_ROOT=/path/to/output/timewindow_verify \
csh cmd/validate_timewindow_training_pro.csh
```

**通过条件：**

1. 命令返回码为 0；
2. `OUTPUT_ROOT/STATUS.txt` 最后一项包含 `DONE`；
3. `OUTPUT_ROOT/logs/` 下存在 Step1–Step6 日志；
4. `step6_retrain/` 下生成最终重训与评估产物。

**注意：**验证脚本会运行 Step1–Step6，而不只是检查 Python 导入。首次执行前必须确认输入目录和输出目录。建议为每次验证使用新的 `OUTPUT_ROOT`。

## 2.3 完整启用顺序

1. 确认目标链路：从已有 cache 训练，或从原始 FSDB 前处理；
2. 检查操作系统、shell、存储、调度器和访问权限；
3. 安装或迁移 Python 环境；
4. 若使用 FSDB 链路，配置并验证 FsdbReader、Verdi 和 license；
5. 设置路径、输入、输出及调度参数；
6. 执行静态检查和小规模验证；
7. 验证通过后再提交正式作业。

# 第三章　安装与部署

## 3.1　基础要求

| 项目 | 要求 |
| --- | --- |
| 操作系统 | Linux x86_64。 |
| Shell | 必须能够执行 `csh`；交互环境可使用 Bash/Sh 或 Tcsh。 |
| Python | 当前已验证环境使用 Python 3.8.17；正式运行建议使用固定版本的独立环境。 |
| 编译工具 | FSDBAPI 需要支持 C++17 的 `g++` 和 `make`。 |
| 训练调度器 | 主训练脚本默认通过 `dsub` 提交任务。 |
| FSDB 调度器 | FSDB 前处理脚本通过 `bsub` 提交任务。 |
| 存储 | 正式流程建议预留 TB 级空间；验证环境可按实际数据规模降低。 |
| 内存 | 正式流程需根据特征数和采样量估算；600 GB 是大规模训练参考值，不是安装最低要求。 |

## 3.2　Python 环境部署

### 3.2.1　方式一：使用已有环境

若机器上已有经过验证的环境，直接记录解释器绝对路径：

```
export CONDA_ENV_ROOT=/path/to/conda/envs/pro
export PYTHON="${CONDA_ENV_ROOT}/bin/python"
"${PYTHON}" --version
```

当前脚本未显式设置 `PYTHON` 时会回退到开发环境的绝对路径。迁移到其他机器时，必须在启动命令前覆盖 `PYTHON`，不要依赖该默认值。

### 3.2.2　方式二：迁移 conda-pack 环境

在已验证环境所在机器执行：

```
conda install -c conda-forge conda-pack
conda pack -n pro -o pro-conda-env.tar.gz
```

将压缩包传输到目标机器后执行：

```
mkdir -p /path/to/conda/envs/pro
tar -xzf pro-conda-env.tar.gz -C /path/to/conda/envs/pro
/path/to/conda/envs/pro/bin/conda-unpack
```

然后设置：

```
export CONDA_ENV_ROOT=/path/to/conda/envs/pro
export PYTHON="${CONDA_ENV_ROOT}/bin/python"
```

批处理作业通常不要求先执行 `condaactivate`；只要 `PYTHON` 指向正确解释器即可。交互调试或依赖 Conda 修改 `PATH` 的外部命令可按需激活环境。

### 3.2.3 方式三：重新创建环境

仓库当前没有受版本管理的 `environment.yml`、requirements 锁定文件或完整的离线包清单。因此，从零安装只能作为开发或临时验证方案，不能声明与当前 `pro` 环境完全一致。

基础依赖至少包括：

```
python -m pip install \
  numpy scipy pandas scikit-learn xgboost \
  optuna matplotlib joblib
```

MCP 路径推荐安装与现有代码兼容的 `skglm`。如果缺失，Step2 MCP 实现可能进入 variance fallback；正式结果复现前必须确认实际选择器。

后续应补充机器可读、可审查的锁定文件，并把它作为依赖版本的唯一来源。

## 3.3 Python 依赖验证

```
"${PYTHON}" - <<'PY'
import sys
import numpy, scipy, sklearn, pandas, xgboost, optuna
import matplotlib, joblib

print("python", sys.executable)
print("numpy", numpy.__version__)
print("scipy", scipy.__version__)
print("sklearn", sklearn.__version__)
print("pandas", pandas.__version__)
print("xgboost", xgboost.__version__)
print("optuna", optuna.__version__)
print("matplotlib", matplotlib.__version__)
print("joblib", joblib.__version__)

try:
    import skglm
```

```
    print("skglm", getattr(skglm, "__version__", "unknown"))
except ImportError:
    print("WARNING: skglm missing; MCP may use fallback")
PY
```

## 3.4 FSDBAPI 与 FsdbReader 部署

FSDBAPI 位于 `PACKAGE_ROOT/src/fsdbapi`。Makefile 优先使用工程内置目录：

```
src/fsdbapi/third_party/verdi/FsdbReader
```

若该目录存在 `ffrAPI.h`，执行：

```
cd "${PACKAGE_ROOT}/src/fsdbapi"
make
```

若需要使用外部 Verdi/FsdbReader：

```
cd "${PACKAGE_ROOT}/src/fsdbapi"
make VERDI_HOME=/path/to/verdi/R-2020.12-SP2
```

切换 FsdbReader 版本或安装目录后应重新编译，不建议直接移动旧二进制。编译完成后检查：

```
test -x build/fsdb_to_tog_cache
test -x build/fsdb_to_power_label
ldd build/fsdb_to_tog_cache | grep 'not found'
```

最后一条命令没有输出，表示 `ldd` 未发现缺失动态库。仍需使用实际 FSDB 做最小读取验证，才能确认 license 和文件兼容性。

# 第四章　配置

## 4.1　路径模型

本文档采用以下互不混用的路径变量：

| 变量 | 示例 | 说明 |
| --- | --- | --- |
| PACKAGE_ROOT | /work/xgboost_timewindow | 软件包根目录，包含 cmd/、src/ 和 docs/。 |
| PROJECT_ROOT | /work | 工作区根目录；脚本默认取 PACKAGE_ROOT 上一级。 |
| CONDA_ENV_ROOT | /opt/conda/envs/pro | Conda 环境目录，不属于软件包目录。 |
| PYTHON | CONDA_ENV_ROOT/bin/python | 启动 Python 程序的解释器。 |
| OUTPUT_ROOT | /data/output/run_001 | 本次训练的独立输出目录。 |
| WORK_DIR | /data/tmp/tw_work | 临时文件和任务工作目录。 |

推荐在每次运行前明确记录：

```
export PACKAGE_ROOT=/path/to/xgboost_timewindow
export PROJECT_ROOT=/path/to/workspace
export CONDA_ENV_ROOT=/path/to/conda/envs/pro
export PYTHON="${CONDA_ENV_ROOT}/bin/python"
```

## 4.2　训练链路必填配置

| 变量 | 要求 | 说明 |
| --- | --- | --- |
| PYTHON | 必须确认 | 已安装完整依赖的 Python 解释器。 |
| LEVEL_DIR | 必须存在 | 降维后的 level cache 目录。 |
| TOG_DIR | 必须存在 | 降维后的 tog cache 目录。 |
| LABEL_DIR | 必须存在 | Power label 的 NPY 文件目录。 |
| NAMEFILE | 必须存在 | 与 cache 同批生成的 signal name 文件。 |

| | | |
|---|---|---|
| OUTPUT_ROOT | 强烈建议设置 | 本次运行输出目录，应使用新目录或明确可续跑目录。 |

**注意：** LEVEL_DIR、TOG_DIR 和 NAMEFILE 必须来自同一批降维结果。不要把不同 run、不同 stage 或不同硬件的数据混合使用。

## 4.3 训练调度配置

| 变量 | 默认值 | 说明 |
|---|---|---|
| DSUB_CMD | dsub | 训练任务提交命令。 |
| DSUB_QUEUE | 空 | 留空时使用调度器默认队列。 |
| DSUB_EXTRA_ARGS | 空数组 | 额外资源参数；C Shell 数组语法。 |
| POLL_SEC | 30 | 主流程检查状态文件的轮询间隔。 |
| STEP1_PARALLEL_DSUB | 1 | Step1 是否拆分提交。 |
| STEP2_PARALLEL_DSUB | 1 | Step2 是否拆分提交。 |

执行正式训练前至少确认：

```
command -v dsub
dsub --help
```

若站点 dsub 不支持 --help，使用该站点规定的只读查询命令确认客户端可用。

## 4.4 FSDB 链路配置

cmd/run_fsdb_pipeline.csh 当前在脚本中定义 fsdb_list、power_list、采样参数和层级过滤参数。启动前必须审查以下内容：

- fsdb_list：workload 名、RTL FSDB、起止时间；
- power_list：与 workload 对齐的 power FSDB 和 signal；
- STEP、RAW_COL、MAX_MEM、TARGET_COLS；
- SIGNAL_PREFIX 和 MAX_CYCLES_PER_FILE；
- BSUB_QUEUE、BSUB_EXTRA_ARGS、POLL_SEC。

**注意：** FSDB pipeline 启动时会重建 WORK_DIR/cache_HW，并清理部分 complete/failed 标志。不要让 WORK_DIR 或 OUTPUT_BASE 指向需要保留且未备份的其他运行目录。

## 4.5 站点激活文件

FSDB pipeline 会在包根目录查找可选的 activate.csh。该文件适合集中维护站点变量：

```
# ${PACKAGE_ROOT}/activate.csh 示例
setenv VERDI_HOME /path/to/verdi/R-2020.12-SP2
setenv FSDB_READER_DIR ${VERDI_HOME}/share/FsdbReader
setenv SNPSLMD_LICENSE_FILE /path/or/license-server

if ( $?LD_LIBRARY_PATH ) then
    setenv LD_LIBRARY_PATH ${FSDB_READER_DIR}/linux64:${LD_LIBRARY_PATH}
else
    setenv LD_LIBRARY_PATH ${FSDB_READER_DIR}/linux64
endif
```

真实 license 信息和服务器地址不应提交到公共仓库。若二进制已通过 rpath 正确定位动态库，不需要重复设置 `LD_LIBRARY_PATH`。

# 第五章　操作指南

## 5.1　启动已有 cache 训练

**目的：**从降维后的 level/tog cache 运行完整 Step1–Step6 主流程。

**前置条件：**第 6 章 L0 和 L1 检查均通过，`dsub` 可用，输出目录可写。

```
cd /path/to/xgboost_timewindow

PYTHON=/path/to/conda/envs/pro/bin/python \
LEVEL_DIR=/path/to/reduced/level \
TOG_DIR=/path/to/reduced/tog \
LABEL_DIR=/path/to/power_labels \
NAMEFILE=/path/to/reduced/level/after_one.namefile.txt \
OUTPUT_ROOT=/path/to/output/timewindow_run_001 \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

第二个位置参数支持：

| 模式 | 行为 |
|---|---|
| `all` | 运行完整流程。 |
| `prep` | 运行准备和筛选阶段，具体阶段边界以入口脚本为准。 |
| `train_k` | 对指定 K 执行训练；第三个位置参数必须提供 K。 |

指定 K 的示例：

```
csh cmd/run_timewindow_pipeline.csh gemmini train_k 1024
```

## 5.2　选择代理筛选方式

默认 `PROXY_SELECTOR=mcp`。使用 MCP：

```
PROXY_SELECTOR=mcp \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

使用 XGBoost proxy 推荐调用封装入口：

```
PYTHON=/path/to/conda/envs/pro/bin/python \
XGB_PROXY_TOP_K=200 \
```

```
XGB_PROXY_ROUNDS=50 \
csh cmd/run_timewindow_pipeline_xgb_proxy.csh gemmini all
```

选择器的算法、参数含义和输出契约见 `docs/timewindow_training_workflow.md`，本指南不重复说明。

## 5.3 启动 FSDB 前处理

**目的：**从 RTL FSDB 和 power FSDB 生成降维后的 level/tog cache 与标签。

**前置条件：**

1. 已检查并按本次数据修改 `fsdb_list` 与 `power_list`；
2. FSDBAPI 已编译，动态库和 license 可用；
3. `bsub` 可用；
4. 工作目录和输出目录有足够空间且不包含需要保留的同名临时 cache。

执行：

```
cd /path/to/xgboost_timewindow
csh cmd/run_fsdb_pipeline.csh gemmini
```

通过条件：终端输出 “FSDB Pipeline 完成”，`OUTPUT_BASE/complete` 存在，且对应 `HW_reduced/level`、`HW_reduced/tog` 目录包含 cache 和 signal name 文件。

## 5.4 查询运行状态

训练主流程将日志和提交脚本写入：

```
${OUTPUT_ROOT}/logs_dsub/
${OUTPUT_ROOT}/jobs_dsub/
```

各阶段使用 `complete`、`failed` 和 `status` 文件协调依赖。排查时先确认失败阶段，再查看该阶段对应日志；不要只根据主进程是否仍在轮询判断任务成功。

快速验证脚本使用：

```
${OUTPUT_ROOT}/STATUS.txt
${OUTPUT_ROOT}/logs/
```

FSDB 前处理主要日志位于：

```
${OUTPUT_BASE}/logs/
${OUTPUT_BASE}/${HW}_reduced/complete
${OUTPUT_BASE}/${HW}_reduced/failed
```

## 5.5 续跑与重新运行

- 新实验优先使用新的 `RUN_TAG` 或 `OUTPUT_ROOT`；
- `STEP1_SKIP_EXISTING=1` 会复用已有 Step1 产物；
- 续跑前确认输入、offset、data type、stage 和代理选择器没有变化；
- 输入或关键配置变化时，不应复用旧 manifest、buffer 或状态文件；
- 清理或删除任何输出前，先确认路径和备份策略。

# 第六章　验证与验收

## 6.1　L0：静态环境检查

在提交任务前执行：

```
test -d "${PACKAGE_ROOT}/cmd"
test -x "${PYTHON}"
command -v csh
command -v dsub
"${PYTHON}" -c \
  "import numpy,scipy,sklearn,pandas,xgboost,optuna"
```

使用 FSDB 链路时增加：

```
command -v bsub
test -x "${PACKAGE_ROOT}/src/fsdbapi/build/fsdb_to_tog_cache"
test -x "${PACKAGE_ROOT}/src/fsdbapi/build/fsdb_to_power_label"
ldd "${PACKAGE_ROOT}/src/fsdbapi/build/fsdb_to_tog_cache" \
  | grep 'not found'
```

前五项应返回 0；最后一项应无输出。无输出只表示动态链接检查通过，不代表 license 和 FSDB 文件读取必然成功。

## 6.2　L1：输入与权限检查

```
test -d "${LEVEL_DIR}"
test -d "${TOG_DIR}"
test -d "${LABEL_DIR}"
test -f "${NAMEFILE}"
test -r "${NAMEFILE}"
mkdir -p "${OUTPUT_ROOT}"
test -w "${OUTPUT_ROOT}"
```

同时人工确认：

1. level、tog、label 中的 case/stem 能够对应；
2. namefile 与 `STAGE` 一致；
3. 输出目录不是其他实验正在使用的目录；

4. 临时目录和输出磁盘空间满足本次配置。

## 6.3 L2：小规模端到端验证

使用第 2.2 节命令运行 `validate_timewindow_training_pro.csh`。为降低验证规模，可设置：

```
SAMPLE_ROWS_PER_FILE=128 \
SAMPLE_BINS=16 \
SAMPLE_MIN_PER_BIN=2 \
PYTHON=/path/to/conda/envs/pro/bin/python \
csh cmd/validate_timewindow_training_pro.csh
```

该验证会依次运行 Step1–Step6。若任一步返回非零状态，脚本会退出；应从 `STATUS.txt` 的最后一个 start/done 记录和对应日志定位失败步骤。

## 6.4 验收标准

| 验收项 | 通过条件 |
|---|---|
| Python 环境 | 解释器路径正确，必需依赖导入成功；MCP 是否使用 `skglm` 已确认。 |
| FSDBAPI | 二进制存在、动态库无缺失，最小 FSDB 读取成功。 |
| 输入数据 | level/tog/label/namefile 可读且属于同批数据。 |
| 调度器 | `dsub` 或 `bsub` 可提交到目标队列并写出日志。 |
| 训练验证 | `STATUS.txt` 包含 `DONE`，Step1–Step6 产物完整。 |
| FSDB 流程 | `HW_reduced/complete` 和总体 `complete` 存在，无对应 `failed`。 |
| 可追溯性 | 日志中可以确认 Python、输入、输出、选择器和关键参数。 |

# 第七章　故障排查

## 7.1　Python 解释器或依赖失败

**现象：**提示解释器不存在，或 `ModuleNotFoundError`。

**诊断：**

```
ls -l "${PYTHON}"
"${PYTHON}" -c "import sys; print(sys.executable); print(sys.path)"
```

**处理：**重新设置 `PYTHON`；若为 conda-pack 环境，确认已运行 `conda-unpack`。不要通过修改系统 Python 临时掩盖项目环境问题。

## 7.2　MCP 结果与预期不一致

**现象：**MCP 阶段进入 variance fallback，或筛选结果与已验证环境不同。

**诊断：**

```
"${PYTHON}" -c "import skglm; print(skglm.__file__)"
```

**处理：**安装与项目兼容的 `skglm`，或明确改用 XGBoost proxy。正式结果中应记录实际使用的选择器和 fallback 状态。

## 7.3　FsdbReader 动态库失败

**现象：**提示 `libnffr.so`、`libnsys.so` 等无法加载。

**诊断：**

```
ldd src/fsdbapi/build/fsdb_to_tog_cache | grep 'not found'
```

**处理：**优先使用正确的 `VERDI_HOME` 或 `FSDB_READER_DIR` 重新编译。仅在临时诊断时设置 `LD_LIBRARY_PATH`，并确认路径对应同一 FsdbReader 版本。

## 7.4　Verdi license 失败

**现象：**二进制能够加载，但打开 FSDB 或调用库时报告 license 错误。

**处理：**检查站点规定的 license 环境变量、服务器连通性和授权范围。license 地址属于站点敏感配置，应由管理员提供，不写入公开文档或仓库。

## 7.5 调度任务提交失败

**现象：** dsub 或 bsub 返回非零状态，任务没有进入队列。

**诊断：** 检查命令是否存在、队列名、资源参数、账户权限以及提交日志。

**处理：** 使 DSUB_QUEUE/DSUB_EXTRA_ARGS 或 BSUB_QUEUE/BSUB_EXTRA_ARGS 与当前集群策略一致。注意两条链路使用不同调度器，不要混用变量名。

## 7.6 输入数据或 namefile 不匹配

**现象：** 列索引越界、feature name 数量不符、case/stem 缺失或 manifest 校验失败。

**处理：** 重新确认 level、tog、label、namefile 和 stage 的来源。输入来源变化后使用新的 OUTPUT_ROOT，避免复用旧 manifest 和 buffer。

## 7.7 已有产物被错误复用

**现象：** 参数修改后任务快速跳过，或结果仍表现为旧配置。

**处理：** 检查 STEP1_SKIP_EXISTING、各阶段 complete 文件和输出目录。优先新建 run 目录；如果必须清理旧产物，应先确认精确路径并备份需要保留的结果。

## 7.8 内存或磁盘不足

**处理：** 验证阶段降低 SAMPLE_ROWS_PER_FILE 或 SAMPLE_ROWS_TOTAL；正式任务结合 TARGET_COLS、cache 分片、Step1 输出压缩和调度器资源参数重新估算。清理前先区分可再生临时文件和需要归档的训练结果。

# 附录 A　常用环境变量参考

## A.1　训练路径与输入

| 变量 | 当前默认或来源 | 说明 |
|---|---|---|
| PACKAGE_ROOT | 脚本自动定位 | 软件包根目录。 |
| PROJECT_ROOT | PACKAGE_ROOT/.. | 工作区根目录。 |
| PYTHON | 开发机绝对路径 | 迁移后必须覆盖。 |
| REDUCTION_BASE | PROJECT_ROOT/output/... | 默认降维输出根目录。 |
| LEVEL_DIR | 按 HW 推导 | Level cache。 |
| TOG_DIR | 按 HW 推导 | TOG cache。 |
| LABEL_DIR | 按 HW 推导 | Power label。 |
| STAGE | after_one | 降维阶段。 |
| NAMEFILE | 按 LEVEL_DIR/STAGE 推导 | Signal name 映射。 |
| RUN_TAG | 当前时间戳 | 运行标识。 |
| OUTPUT_ROOT | 按 HW/STAGE/RUN_TAG 推导 | 本次训练目录。 |

## A.2　常用训练控制变量

| 变量 | 当前默认 | 说明 |
|---|---|---|
| OFFSETS | 12,11,10,9,8,7,6 | Time-window 偏移。 |
| DATA_TYPES | tog,level | 输入类型。 |
| TEST_COUNT | 1 | 测试 case 数量。 |
| SAMPLE_ROWS_PER_FILE | 0 | 每文件采样量；0 表示不按文件采样。 |
| SAMPLE_ROWS_TOTAL | 0 | 全局采样总量；0 表示不启用。 |

| | | |
|---|---|---|
| SAMPLE_BINS | 20 | Power 分层采样 bin 数。 |
| SAMPLE_SEED | 108 | 采样随机种子。 |
| PROXY_SELECTOR | mcp | mcp、xgb 或 xgboost。 |
| TOP_KS | 256,512,1024, 2048,4096 | Screening 和后续训练的 K。 |
| RUN_STEP2_VALIDATION | 1 | 是否执行 Step2 独立验证。 |

# 附录 B　命令速查

## B.1　Python 依赖检查

```
"${PYTHON}" -c \
  "import numpy,scipy,sklearn,pandas,xgboost,optuna"
```

## B.2　快速训练验证

```
PYTHON=/path/to/python \
LEVEL_DIR=/path/to/level \
TOG_DIR=/path/to/tog \
LABEL_DIR=/path/to/labels \
NAMEFILE=/path/to/after_one.namefile.txt \
OUTPUT_ROOT=/path/to/new/verify_output \
csh cmd/validate_timewindow_training_pro.csh
```

## B.3　正式训练

```
PYTHON=/path/to/python \
LEVEL_DIR=/path/to/level \
TOG_DIR=/path/to/tog \
LABEL_DIR=/path/to/labels \
NAMEFILE=/path/to/after_one.namefile.txt \
OUTPUT_ROOT=/path/to/new/train_output \
csh cmd/run_timewindow_pipeline.csh gemmini all
```

## B.4　FSDB 前处理

```
cd /path/to/xgboost_timewindow
csh cmd/run_fsdb_pipeline.csh gemmini
```

# 附录 C　术语与缩略语

| 术语 | 说明 |
|---|---|
| Package root | `xgboost_timewindow` 软件包根目录。 |
| Project root | 软件包上一级工作区目录。 |
| Cache | 从 FSDB 导出并经清理、降维后的稀疏特征数据。 |
| Namefile | 将特征列索引映射回 signal 名称的文件。 |
| Manifest | 描述输入文件、组合、索引或中间产物的元数据文件。 |
| Smoke test | 使用较小配置验证环境与关键链路的端到端测试。 |
| MCP | Minimax Concave Penalty，本项目中的一种代理筛选方式。 |